Data Mining and Machine Learning

Gennaro Basile

March 23, 2026

Contents

1 Introduction to Artificial Neural Networks **3**

1.1 Recap: from Perceptrons to ANNs . . . 3

1.2 Multi-Layer Perceptron (MLP) Architecture . . . 3

1.2.1 A Simple Layout . . . 3

1.2.2 The Mathematics of Data Flow (Forward Propagation) . . . 4

1.2.3 The Network as a Function Composition . . . 4

1.2.4 Network Design: Parameters vs. Hyperparameters . . . 5

1.2.5 Universal Approximation Theorems . . . 5

1.2.6 The Power of Hidden Layers: Feature Hierarchies . . . 5

1.3 Analyzing a Simple MLP: The XOR Forward Pass . . . 6

1.3.1 The XOR Problem Revisited . . . 6

1.3.2 Network Architecture & Dimensionality . . . 6

1.3.3 The Forward Pass: Step-by-Step . . . 7

1.3.4 Why This Example Is Important . . . 8

1.4 Backpropagation: The Backward Pass . . . 8

1.4.1 The Goal of Backpropagation . . . 8

1.4.2 The Loss Function and Regularization . . . 8

1.5 The XOR Operator and General MLP Terminology . . . 9

1.5.1 Why XOR Proves the Need for MLPs . . . 9

1.5.2 The Multi-Layer Perceptron (MLP) . . . 9

1.5.3 The Knowledge Base: Managing Data in Machine Learning . . . 10

1.6 The Supervised Learning Workflow . . . 10

1.6.1 The Machine Learning Pipeline . . . 10

1.6.2 Case Study: The MNIST Digit Classifier . . . 11

1 Introduction to Artificial Neural Networks

The lecture introduces the Multi-Layer Perceptron (MLP), covering its basic architecture, the algebraic description of feed-forward networks, the Universal Approximation Theorems, and the role of hidden layers in learning hierarchical representations.

1.1 Recap: from Perceptrons to ANNs

In previous lectures we saw that a single **perceptron** can implement simple logic gates. However, using perceptrons directly for complex computations is inefficient: every new task requires manual, ad-hoc tuning of the weights and biases.

The key insight is that we can build **learning algorithms**, i.e. Artificial Neural Networks (ANNs), that automatically adjust the weights and biases of a network of artificial neurons. This adjustment happens entirely in response to external data, with no direct human intervention at the level of individual weights.

Three ingredients were necessary before real progress could be made:

1. **Non-linearity.** For a network to *learn*, a small change in a weight must produce a correspondingly small change in the output. Step functions (as used in the original perceptron) do not satisfy this requirement, because their output jumps abruptly. Smooth, differentiable activation functions (such as the sigmoid) solve this problem.
2. **Backpropagation.** An efficient algorithm that computes the gradient of the loss function with respect to every weight in the network, enabling systematic weight updates.
3. **Computational power.** Training large networks requires performing billions of floating-point operations, which only became feasible with modern hardware (GPUs, TPUs).

1.2 Multi-Layer Perceptron (MLP) Architecture

1.2.1 A Simple Layout

Feed-Forward Neural Network

A **feed-forward neural network** (also called a Multi-Layer Perceptron, MLP) is a network of artificial neurons organized in **layers**, where information flows in one direction only: from the **input layer**, through one or more **hidden layers**, to the **output layer**. Neurons within the same layer are *not* connected to each other; each neuron connects only to neurons in the immediately subsequent layer.

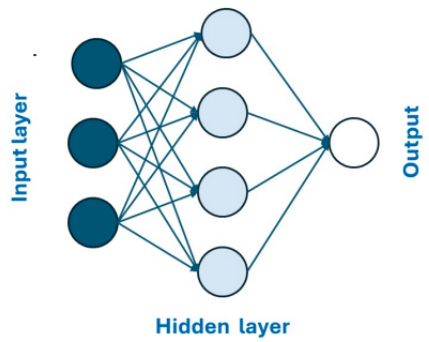

Figure 1.2.1: *Feed Forward Network*

In a diagram each circle represents a neuron and each line represents a weighted connection. The key properties of this layout are:

- **Input layer:** receives the raw data (e.g. pixel values of an image). Each neuron holds one component of the input vector.
- **Hidden layer(s):** process the information coming from the previous layer. Each successive hidden layer builds higher-level, more abstract representations of the data.
- **Output layer:** produces the final prediction (e.g. a class label).

An important detail: the output of each neuron is *unique*: that is, a neuron produces a single scalar value, and that same value is sent to *all* neurons in the next layer.

1.2.2 The Mathematics of Data Flow (Forward Propagation)

A fully connected neural network transforms input data layer by layer. Regardless of how deep the network is, every single hidden layer performs the exact same two-step mathematical operation.

Let L^h be a hidden layer, $\mathbf{a}^{(h)}$ be its activation vector, $\mathbf{W}^{(h)}$ the weight matrix connecting it to the next layer, and $\mathbf{b}^{(h)}$ the bias vector.

Step 1: Pre-Activation (Linear Transformation) . For the next layer $h+1$, we first compute the **pre-activation vector** ($\mathbf{z}$). This is simply the dot product of the weights and the previous layer's activations, plus the bias:

$$\mathbf{z}^{(h+1)} = \mathbf{W}^{(h)}\mathbf{a}^{(h)} + \mathbf{b}^{(h)} \tag{1}$$

Step 2: Activation (Non-Linearity) . We then apply a differentiable, non-linear **activation function** (σ) element-wise to the pre-activation vector to get the final **activation vector** ($\mathbf{a}$):

$$\mathbf{a}^{(h+1)} = \sigma\Big(\mathbf{z}^{(h+1)}\Big) \tag{2}$$

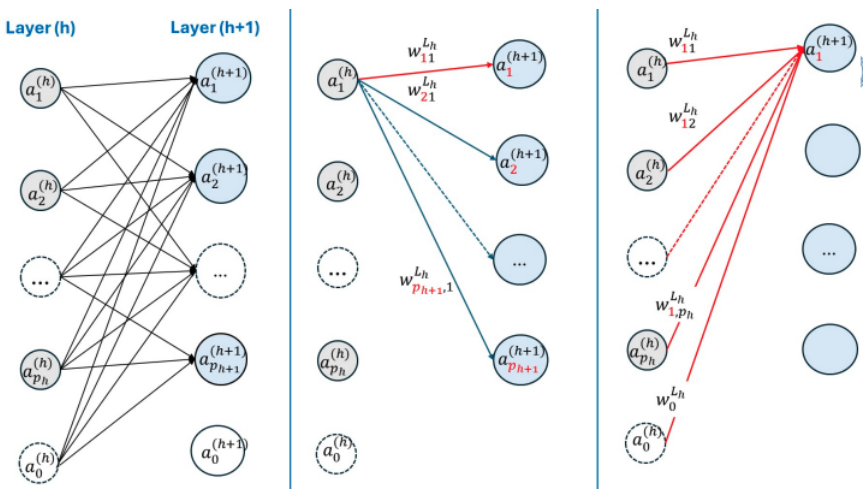

Figure 1.2.2: *Neural Network scheme*

1.2.3 The Network as a Function Composition

Because every layer performs these same two steps, an entire deep neural network can be written as one massive nested mathematical function. If $\mathbf{x}$ is our input data, the final output $\mathbf{O}$ is:

$$\mathbf{O} = \sigma_L\Big(\ldots\sigma_2(\mathbf{W}^{(2)}\sigma_1(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)})\ldots\Big) \tag{3}$$

Why is the Activation Function (σ) strictly necessary?
If we removed the non-linear σ, the network would just be a series of matrix multiplications: $\mathbf{W}^{(3)}(\mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x}))$. In linear algebra, multiplying multiple matrices just results in one single new matrix. Therefore, without non-linearity, a 100-layer deep neural network mathematically collapses into a basic linear regression model, entirely losing its ability to learn complex, curved boundaries.

1.2.4 Network Design: Parameters vs. Hyperparameters

When building these networks, a Data Scientist must clearly distinguish between what the machine learns and what the human decides:

- **Parameters (Learned by the machine):** The actual numerical values inside the weight matrices ($\mathbf{W}$) and bias vectors ($\mathbf{b}$). These are automatically adjusted by the Backpropagation algorithm during training.
- **Hyperparameters (Set by the human):** The architectural design choices made *before* training begins. This includes the number of hidden layers, the number of neurons in each layer, the learning rate, and the choice of activation functions (e.g., ReLU vs. Sigmoid).

1.2.5 Universal Approximation Theorems

These theorems provide a mathematical justification for using neural networks, assuring researchers that a sufficiently large or deep network can model the complex, non linear relationships often found in real-world data.

Universal Approximation Theorem (Hornik et al., 1988)

A feed-forward neural network with a single hidden layer and a non-polynomial activation function (e.g. sigmoid, ReLU) can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to any desired degree of accuracy, provided the hidden layer has sufficiently many neurons.

There are two flavors of this result:

1. **Width approach:** keep the network shallow (one hidden layer) but make it *wider* by increasing the number of neurons in that layer.
2. **Depth approach:** keep the width fixed but make the network *deeper* by adding more hidden layers.

Both strategies can achieve universal approximation in theory.

These are **existence theorems**. They guarantee that a network with the right structure *exists*, but they say nothing about:

- how many neurons/layers are actually needed,
- how to find the correct values of the weights.
- whether gradient-based optimization will converge to the optimal solution.

In practice, finding good weights is the hard part and that is the job of training algorithms like backpropagation combined with gradient descent.

1.2.6 The Power of Hidden Layers: Feature Hierarchies

Why do we need hidden layers at all? The answer lies in the concept of **hierarchical abstraction**.

Hierarchical Feature Learning

Hidden layers allow the network to learn a **hierarchy of representations** from the data. Layers closer to the input learn simple, low-level features (e.g. edges, basic shapes), while layers closer to the output learn complex, high-level features (e.g. object parts, entire objects).

This principle applies across different data modalities:

- **Images:** an image is composed of objects, which are composed of parts, which are composed of edges and textures. Early layers detect edges; middle layers detect shapes (e.g. loops, strokes); later layers detect complete objects (e.g. digits).
- **Text:** a sentence is composed of words, which are composed of syllables, which are composed of characters. Each level follows specific patterns that the network can learn.
- **Sound:** audio is composed of musical phrases, which are composed of notes organized at different levels.

1.3 Analyzing a Simple MLP: The XOR Forward Pass

1.3.1 The XOR Problem Revisited

The XOR function takes two binary inputs and returns:

- 1 if the two inputs are different,
- 0 if the two inputs are equal.

Its truth table is:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

As previously established, a single linear perceptron cannot solve the XOR logic gate because the data is not linearly separable. To solve it, we must build a **Multi-Layer Perceptron (MLP)** with at least one hidden layer.

1.3.2 Network Architecture & Dimensionality

Understanding matrix dimensions is the most critical skill for coding neural networks. Our XOR network is structured as follows:

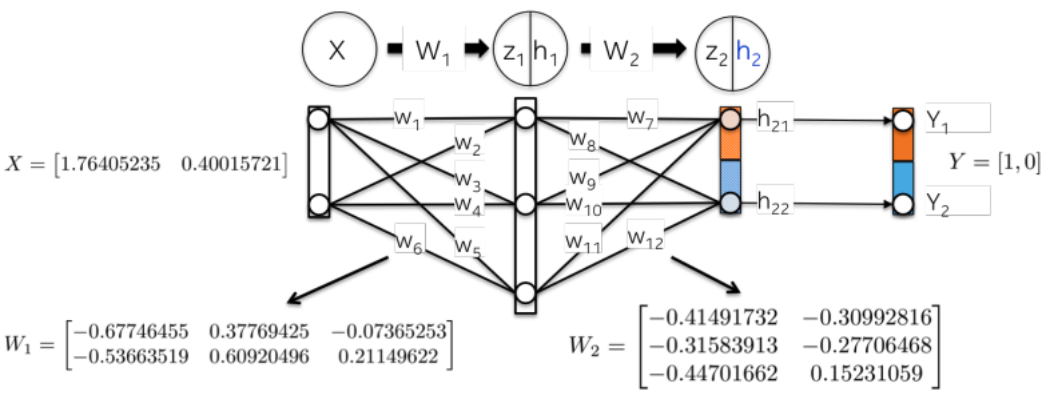

Figure 1.3.1: *Schematic layout of MLP*

- **Input (X):** 1 sample with 2 features. Dimension = (1×2).
- **Hidden Layer (h_1):** We choose 3 neurons. To get from 2 inputs to 3 neurons, our first weight matrix (W_1) must have the dimension (2×3).
- **Output Layer (h_2):** We need 2 outputs (for our one-hot classes). To get from 3 hidden neurons to 2 outputs, our second weight matrix (W_2) must be (3×2).

1.3.3 The Forward Pass: Step-by-Step

We will trace a single input vector $X = [1.76, 0.40]$ through the network to see how it transforms. (Note: Initial weights are chosen randomly).

Step 1: Input to Hidden Layer (Linear) . We calculate the pre-activation z_1 by multiplying the input by the first weight matrix:

$$z_1 = X \cdot W_1$$

$$(1 \times 2) \cdot (2 \times 3) \rightarrow \text{Resulting dimension is } (1 \times 3)$$

Data Science Takeaway: The network has mathematically projected our 2D input into a new, 3-dimensional internal feature space.

Step 2: Hidden Layer Activation (Non-Linear) . We apply the **Sigmoid Function** $\sigma(x) = \frac{1}{1+e^{-x}}$ element-wise to z_1. This squishes all three values into a range between 0 and 1, providing the crucial non-linearity.

$$h_1 = \sigma(z_1) = [0.19, \quad 0.71, \quad 0.48]$$

Step 3: Hidden to Output Layer (Linear) . We take our new 3D feature vector (h_1) and multiply it by the second weight matrix to calculate the final pre-activation z_2:

$$z_2 = h_1 \cdot W_2$$

$$(1 \times 3) \cdot (3 \times 2) \rightarrow \text{Resulting dimension is } (1 \times 2)$$

The result is $z_2 = [-0.52, \quad -0.18]$. These are our raw output scores (logits).

Step 4: Output Activation (Softmax) . We have two raw scores, but we want probabilities that sum to 100% (1.0). We apply the **Softmax Function**:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

$$h_2 = \text{Softmax}([-0.52, -0.18]) = [0.37, \quad 0.45] \text{ (Simplified for example)}$$

The network took the input $[1.76, 0.40]$ and predicted that it is roughly a 37% match for Class 0, and a 45% match for Class 1.

Because the weights were initialized randomly, this prediction is essentially garbage. To make it accurate, the network would now calculate the Error (Loss) between this prediction and the true One-Hot target, and use **Backpropagation** to adjust W_1 and W_2.

1.3.4 Why This Example Is Important

This example is small, but it shows the essential logic of neural networks:

- the input is multiplied by a weight matrix,
- the result is transformed through a non-linear activation function,
- the hidden representation is passed to another layer,
- the final output is converted into a probability-like vector.

Even though the network is simple, all larger MLPs work in the same general way.

1.4 Backpropagation: The Backward Pass

1.4.1 The Goal of Backpropagation

While the Forward Pass makes a prediction, the **Backward Pass** calculates exactly how wrong that prediction is and updates the network's weights to improve the next prediction.

The algorithm that dictates this process is called **Backpropagation**. It computes the *gradient* of the Loss function with respect to every single weight in the network, moving from the output layer backwards to the input layer.

1.4.2 The Loss Function and Regularization

Before we can backpropagate, we need to mathematically define our error using a **Loss Function** ($\mathcal{L}$). We can use as example Mean Squared Error (MSE) combined with Ridge ($\mathcal{L}^2$) Regularization:

$$\mathcal{L} = \frac{1}{2N}\sum_{i=0}^{N}(y_i - h_{2i})^2 + \frac{\lambda}{2N}\sum_{j=0}^{M} W_j^2$$

- **Data Fitting Term (Left):** Measures the distance between the network's prediction (h_2) and the true label (y).
- **Regularization Term (Right):** Penalizes excessively large weights to prevent overfitting, controlled by the parameter λ.

Step 1: Updating the Output Layer (W_2) . To update the weights connecting the hidden layer to the output layer (W_2), we must find the gradient $\frac{\partial \mathcal{L}}{\partial W_2}$. Because the network is a composite function ($h_1 \to z_2 \to h_2 \to \mathcal{L}$), we use the **Chain Rule**:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial h_2} \cdot \frac{\partial h_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial W_2}$$

1. **Loss Derivative** ($\frac{\partial \mathcal{L}}{\partial h_2}$): How the loss changes with respect to the final prediction.
2. **Activation Derivative** ($\frac{\partial h_2}{\partial z_2}$): The slope of the Softmax/Sigmoid function.
3. **Input Derivative** ($\frac{\partial z_2}{\partial W_2}$): The input to this layer, which is simply the activation of the previous hidden layer (h_1).

Once we multiply these three terms together to get the gradient matrix, we apply **Gradient Descent** to update the weights, stepping in the opposite direction of the gradient by a factor of the learning rate (α):

$$W_2^* = W_2 - \alpha \frac{\partial \mathcal{L}}{\partial W_2}$$

Step 2: Updating the Hidden Layer (W_1) . Now we move backwards to the first weight matrix (W_1). The chain rule gets longer:

$$\frac{d\mathcal{L}}{dW_1} = \frac{d\mathcal{L}}{dh_1}\frac{dh_1}{dz_1}\frac{dz_1}{dW_1}$$

$$\frac{d\mathcal{L}}{dh_1} = \frac{d\mathcal{L}}{dz_2}\frac{dz_2}{dh_1} = \frac{d\mathcal{L}}{dh_2}\frac{dh_2}{dz_2}\frac{dz_2}{dW_2}$$

$$\frac{\partial\mathcal{L}}{\partial W_1} = \left(\frac{\partial\mathcal{L}}{\partial h_2}\cdot\frac{\partial h_2}{\partial z_2}\cdot\frac{\partial z_2}{\partial h_1}\right)\cdot\frac{\partial h_1}{\partial z_1}\cdot\frac{\partial z_1}{\partial W_1}$$

Crucial Data Science Concept: Gradient Reuse
Notice that the first two terms ($\frac{\partial\mathcal{L}}{\partial h_2}\cdot\frac{\partial h_2}{\partial z_2}$) are the exact same derivatives we calculated for W_2. The absolute genius of the Backpropagation algorithm is that **we do not recompute them**. We store the gradients from the later layers and reuse them for the earlier layers. This makes training deep networks computationally efficient.

Once the full gradient is calculated, we update the first matrix just like the second:

$$W_1^* = W_1 - \alpha\frac{\partial\mathcal{L}}{\partial W_1}$$

Conclusion of the Training Step . By updating W_2 and then W_1, the network has completed one full learning cycle. The next time the identical input X is passed forward, the new weights (W_1^* and W_2^*) will produce an output that is slightly closer to the true target Y, resulting in a smaller Loss.

1.5 The XOR Operator and General MLP Terminology

1.5.1 Why XOR Proves the Need for MLPs

As demonstrated in the previous forward-pass example, the **XOR** (Exclusive OR) logical function cannot be solved by a single linear classifier because the classes are not linearly separable.

This classical problem proved that artificial intelligence requires two things to solve complex real-world problems:

1. **Hidden Layers:** To project the input data into a new, higher-dimensional space where the classes *can* be separated.
2. **Non-Linear Activation Functions:** To ensure that the multiple layers do not mathematically collapse back into a single linear equation.

1.5.2 The Multi-Layer Perceptron (MLP)

An MLP is the simplest and most foundational type of Artificial Neural Network (ANN). It is a **Feed-Forward Network**, meaning data only moves in one direction (from left to right) without any loops or feedback cycles.

Structural Terminology

- **Neuron (Node):** The basic computational unit. It takes inputs, multiplies them by weights, adds a bias, and passes the sum through an activation function to produce a single scalar output.

- **Layer:** A vertical column of neurons that operate in parallel. Neurons within the same layer do *not* connect to each other.
- **Hidden Layer:** Any intermediate layer between the input and the output. It is "hidden" because the user never directly sees its values; it creates internal representations of the data.
- **Weights:** The numerical parameters connecting neurons across layers. *These are the unknown variables that the network must learn.*

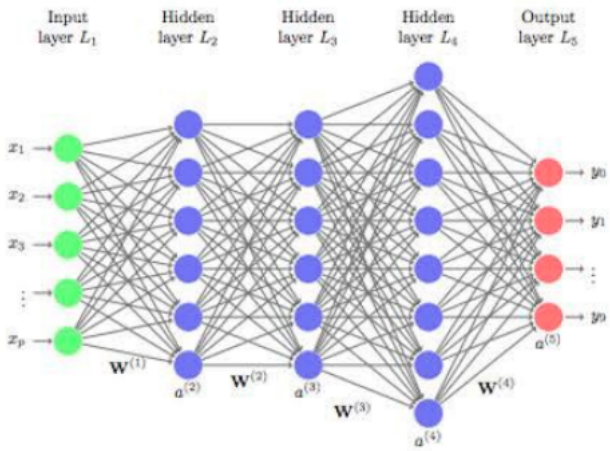

Figure 1.5.1: *Multi Layer Perceptron*

1.5.3 The Knowledge Base: Managing Data in Machine Learning

In Supervised Learning, the network learns from a **Knowledge Base**: a dataset containing input **Features** (X) paired with correct **Targets** (Y).

To properly train and evaluate a model without bias, a Data Scientist must split this knowledge base into three distinct subsets:

1. **Training Set:** The data used to actually teach the model. The network makes predictions on this data, calculates the Cost (Loss) Function, and updates its weights via Backpropagation.
2. **Validation Set:** A separate dataset used *during* development. The model is evaluated on this set to help the data scientist tune **hyperparameters** (like learning rate or number of layers) and check for overfitting. The model does *not* update its weights based on this data.
3. **Test Set:** The final, untouched dataset. It is used only once at the very end of the project to evaluate how well the fully trained model **generalizes** to completely new, unseen data.

The Golden Rule of Data Science: Never, under any circumstances, allow your model to train on or see the Test Set during the development phase.

1.6 The Supervised Learning Workflow

1.6.1 The Machine Learning Pipeline

Supervised learning is not just about the math of a neural network; it is a structured engineering pipeline. The workflow is divided into two distinct phases:

Phase 1: Training (Learning)

1. **Data Prep & Split:** Collect a labeled *Knowledge Base* and split it into a Training Set and a Test Set.
2. **Normalization:** Rescale the input features (e.g., pixel values) to a standard range, usually between 0 and 1. This prevents large numbers from destabilizing the math.
3. **Initialization:** Assign small, random starting values to all weights and biases. (If all weights started at 0, every neuron would learn the exact same thing).
4. **Forward Pass:** Pass a batch of training data through the network to get a prediction.
5. **Cost Calculation:** Use a Cost (Loss) Function to measure how wrong the prediction is compared to the true label.
6. **Backpropagation & Optimization:** Calculate the gradients and update the weights to reduce the error.
7. **Iteration:** Repeat steps 4-6 thousands of times over the training set until a **Stopping Criterion** is met (e.g., the loss stops decreasing).

Phase 2: Inference (Production) Once the stopping criterion is met, training ends. The weights are locked in place, and the model is considered **frozen**. In the Inference phase, the frozen model is exposed to completely new, unseen data (the Test Set or real-world user data) to make final predictions.

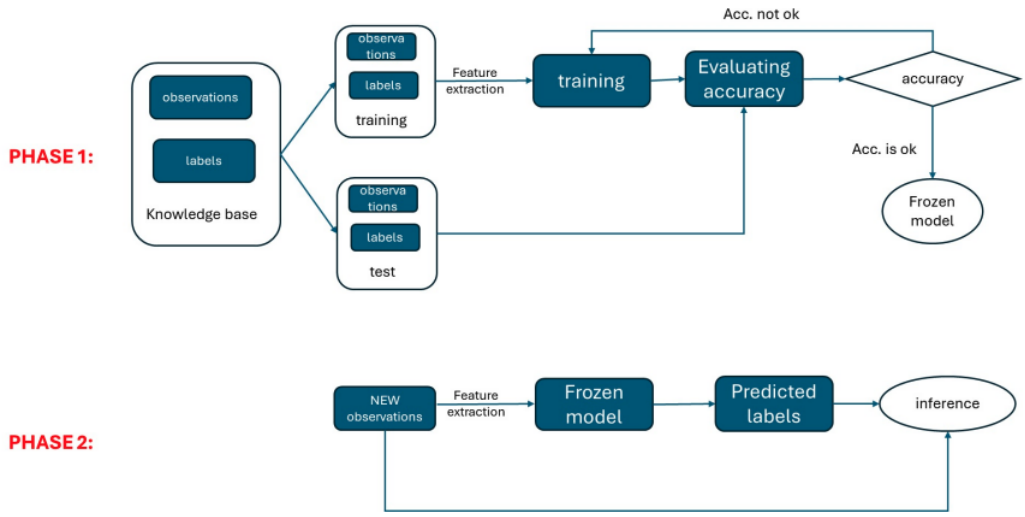

Figure 1.6.1: *Supervised Classification problem*

1.6.2 Case Study: The MNIST Digit Classifier

To make this abstract pipeline concrete, consider the classic "Hello World" of Deep Learning: classifying handwritten digits (0-9) from the MNIST dataset.

Data Representation. Each MNIST image is a 28×28 pixel grayscale image. To feed this into a standard Feed-Forward network, the 2D image is *flattened* into a single 1D vector of 784 pixels.

Network Architecture. The lecture proposes a specific architecture to solve this:

$$784 \text{ (Input)} \rightarrow 16 \text{ (Hidden)} \rightarrow 16 \text{ (Hidden)} \rightarrow 10 \text{ (Output)}$$

- **Input Layer:** 784 neurons, one for each pixel.
- **Hidden Layers:** Two hidden layers with 16 neurons each to learn the shapes and curves of the numbers.

- **Output Layer:** 10 neurons, representing the probability of the image being the digit 0, 1, 2... up to 9. The neuron that "fires" the highest is the network's prediction.

The Scale of Optimization. Even for this incredibly small, simple toy network, the math scales massively.

- Weights between Input and Hidden 1: 784×16
- Weights between Hidden 1 and Hidden 2: 16×16
- Weights between Hidden 2 and Output: 16×10
- Plus the biases for each layer $(16 + 16 + 10)$

Total Parameters to learn: **13,002**.

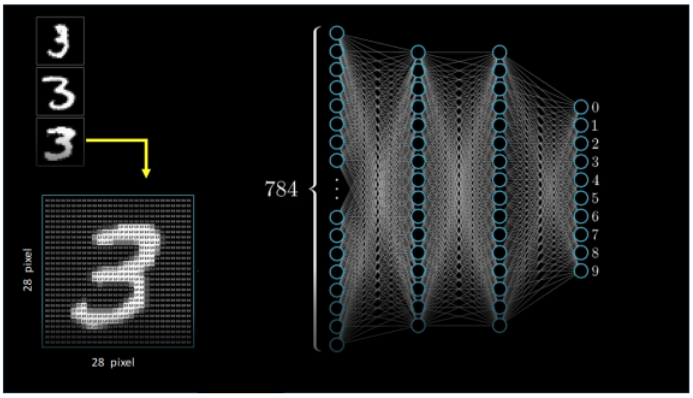

Figure 1.6.2: *The MNIST Digit Classifier*

Training this network means finding the exact configuration of those 13,002 numbers that results in the lowest possible cost function. This is why automated algorithms like Backpropagation and powerful hardware like GPUs are absolutely mandatory for modern AI.